

IL2CPP Automatic Dumper Guide

From Metadata Magic → Version Detection → Full Automatic Dump Pipeline
Connected to the Unity IL2CPP Metadata Deep Dive

0. Connection to Previous Guide

The previous document **IL2CPP_Metadata_Guide.pdf** explained *why* the pattern AF 1B B1 FA 1F 00 00 00 works and how to recover a clean **global-metadata.dat** from memory using Game Guardian.

This document continues from that point. Once you have a clean (or encrypted) metadata file and the corresponding **libil2cpp.so** / GameAssembly.dll, an automatic dumper must:

1. Validate the metadata magic and read the version field
2. Automatically choose the correct structure layout for that version
3. Locate CodeRegistration and MetadataRegistration inside the binary
4. Generate dump.cs, script.json, il2cpp.h, DummyDLL, etc.

Goal of this guide: Understand exactly how automatic version detection works so you can feed any game's il2cpp binary + metadata and get a correct dump without manually guessing the version.

1. Complete Automatic Dump Pipeline

Every modern dumper (Rodroid, Il2CppDumper, Il2CppInspector, Python ports) follows roughly the same pipeline:

Step	Action	What happens
1	Read metadata header	Check magic AF 1B B1 FA → read 4-byte version
2	Version selection	Map version number → structure sizes & offsets
3	Parse metadata tables	Load types, methods, fields, strings using version layout
4	Load binary	Parse ELF / PE / Mach-O / NSO / WASM
5	Find registrations	Locate CodeRegistration + MetadataRegistration
6	Map pointers	Connect method pointers, type definitions, RGCTX
7	Generate outputs	dump.cs, script.json, il2cpp.h, DummyDLL, etc.

2. How Automatic Version Detection Works

The first 8 bytes of a valid global-metadata.dat are critical:

```
Offset 0-3 : AF 1B B1 FA → magic (must be 0xFAB11BAF)
Offset 4-7 : 1F 00 00 00 → version = 31 (little-endian)
```

A good automatic dumper does the following:

- Read the first 4 bytes and verify they equal 0xFAB11BAF. If not → file is encrypted / truncated / wrong.
- Read the next 4 bytes as a signed 32-bit integer → this is the metadata version.
- Look up that version in an internal table of known layouts (16, 19, 20, 21, 22, 24, 24.1–24.5, 27, 27.1, 29, 31, 35, 38, 39, 104, 106...).
- Some dumpers (especially Rodroid) also perform “AutoPlus” refinement: the metadata may claim version 31 while the CodeRegistration layout still matches version 29. They detect this by counting structures and adjust automatically.
- If the version is unknown or the header is corrupted, modern dumpers emit a clear error (exactly the error you saw in Rodroid: “Invalid metadata version field (8311)”).

Key point for automatic dumping: You do **not** need to tell the tool the Unity version manually in most cases. A correct header already contains the version. The dumper only needs a clean metadata file whose first 8 bytes are valid.

3. Practical Version Reference

Version	Hex (LE)	Typical Unity	Notes for auto-dumpers
---------	----------	---------------	------------------------

24	18 00 00 00	2018–2019	Many sub-versions (24.1–24.5)
27	1B 00 00 00	2020–2021	Common, well supported
29	1D 00 00 00	2021–2022	Still widely used
31	1F 00 00 00	2021.3 / 2022.x	Most common today (CPM, etc.)
35+	23+ 00 00 00	Unity 6	New magic possible; Rodroid supports

When the version field itself is garbage (e.g. 8311), the dumper correctly refuses to continue. That almost always means the metadata you supplied is still encrypted or the dump from memory was incomplete.

4. How Rodroid / Modern Dumpers Handle This Automatically

Rodroid (and good Python/C# ports) implement the following logic internally:

```
1. Open global-metadata.dat
2. Read uint32 at offset 0 → must be 0xFAB11BAF
3. Read int32 at offset 4 → version
4. If version is outside known range or header tables are out of bounds → clear error
5. Select the correct Il2CppGlobalMetadataHeader layout for that version
6. (Optional) AutoPlus: refine version by looking at CodeRegistration / MetadataRegistration sizes
7. Parse all tables using the chosen layout
8. Search the binary for the registration structures that match the counts from metadata
9. Generate all output files
```

This is why you only need to supply two files. The tool itself discovers the correct version and structure sizes. Manual “force version” is only needed for heavily protected or broken dumps.

5. Building / Using an Automatic Python Dumper

You have three practical options:

Option A – Use existing Python port (easiest)

Repository: <https://github.com/springmusk026/Il2CppDumper-Python>

```
python -m il2cpp_dumper_py libil2cpp.so global-metadata.dat ./output
```

It already reads the version from the header and selects the layout. Supports v16–31. Slower and less complete than Rodroid, but pure Python.

Option B – Call Rodroid / Rust CLI from Python (best quality)

Download the Rodroid CLI or the il2cpp_dumper binary, then wrap it:

```
import subprocess, pathlib

def auto_dump(il2cpp_path, metadata_path, out_dir="output"):
    pathlib.Path(out_dir).mkdir(exist_ok=True)
    cmd = ["il2cpp_dumper", str(il2cpp_path), str(metadata_path), out_dir]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.returncode == 0, result.stdout, result.stderr
```

Option C – Minimal custom Python (for learning / special cases)

You only need to implement the first two steps yourself for automatic version detection:

```
import struct

def detect_metadata_version(path):
    with open(path, "rb") as f:
        magic, version = struct.unpack("Ii", f.read(8))
        if magic != 0xFAB11BAF:
            raise ValueError(f"Invalid magic: {magic:08X} (expected FAB11BAF)")
        if not (16 <= version <= 106):
            raise ValueError(f"Suspicious version: {version}")
    return version
```

6. Recommended Automatic Workflow for Any Game

- 1. Obtain clean files** — Use Game Guardian (or Frida/Zygisk) to dump decrypted global-metadata.dat. Verify first 8 bytes = AF 1B B1 FA + valid version.

- 2. Obtain matching binary** — libil2cpp.so (Android) or GameAssembly.dll (Windows). Prefer memory dump if the on-disk file is packed.
- 3. Feed both files to an automatic dumper** — Rodroid (best), Python port, or your wrapper. Do not force a version unless the tool asks.
- 4. Check the error message** — If you see “Invalid metadata version field (XXXX)”, the metadata is still bad. Go back to step 1.
- 5. Use the generated dump.cs / script.json** — These are what you need for offset finding, hooking, and mod menus.

This workflow is fully automatic once the two input files are correct. The version is discovered from the metadata header itself — that is the direct connection to the magic bytes explained in the previous PDF.

7. Summary – How Everything Connects

From Metadata Guide	In Automatic Dumper
AF 1B B1 FA magic	First validation check. Wrong magic → refuse dump
1F 00 00 00 = version 31	Automatic layout selection (structure sizes & offsets)
Why search works (Unity forces format)	Why we can rely on the header after a proper memory dump
GG dump of decrypted metadata	Input file that makes automatic version detection possible
Encrypted on-disk file	Causes the “Invalid version field” error you saw in Rodroid

Once you can reliably produce a metadata file that starts with the correct magic + version, any modern automatic dumper (Rodroid, Python port, or a thin Python wrapper) will detect the version and generate a correct dump for almost any Unity IL2CPP game.

Companion to IL2CPP_Metadata_Guide.pdf • Educational / research use only